Assignment
Roll Number: 202512026
Name: Santo Giriso David
Computer programming c++

Here is a complete, standard-compliant C++ program that fulfills all the requirements of your assignment, including file handling (ifstream, ofstream, fstream), CSV formatting, and a menu-driven system using a switch statement.
To handle updates and deletions safely, the program reads the data into a vector of Student objects, modifies the data in memory, and then writes the updated list back to student.txt.

Complete C++ Source Code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

// 1. Requirement: Create a Student Object to store student
information.
class Student {
public:
    string rollNumber;
    string name;
    int age;

    // Helper to format student data as a CSV row
    string toCSV() const {
        return rollNumber + ", " + name + ", " + to_string(age);
    }
};

// Helper function to trim whitespace from strings (useful for clean
CSV parsing)
string trim(const string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

// Helper function to load all students from the file into a vector
vector<Student> loadStudentsFromFile(const string& filename) {
    vector<Student> students;
```

```
    ifstream file(filename);

    if (!file.is_open()) {
        // If file doesn't exist yet, return an empty vector
        return students;
    }

    string line;
    while (getline(file, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string roll, name, ageStr;

        if (getline(ss, roll, ',') && getline(ss, name, ',') &&
getline(ss, ageStr)) {
            Student s;
            s.rollNumber = trim(roll);
            s.name = trim(name);
            s.age = stoi(trim(ageStr));
            students.push_back(s);
        }
    }
    file.close();
    return students;
}

// Helper function to save the entire vector of students back to the
file
void saveStudentsToFile(const string& filename, const vector<Student>&
students) {
    ofstream file(filename, ios::trunc); // Overwrites the file
completely
    if (!file.is_open()) {
        cout << "Error: Could not open file for writing.\n";
        return;
    }
    for (const auto& s : students) {
        file << s.toCSV() << "\n";
    }
    file.close();
}

// 2. Requirement: Implement a function to display all students stored
in the file.
void displayAllStudents(const string& filename) {
    vector<Student> students = loadStudentsFromFile(filename);
```

```
    if (students.empty()) {
        cout << "\nNo student records found.\n";
        return;
    }

    cout << "\n----------------------------------------\n";
    cout << left << setw(15) << "Roll Number" << setw(20) << "Name" <<
setw(5) << "Age" << "\n";
    cout << "----------------------------------------\n";
    for (const auto& s : students) {
        cout << left << setw(15) << s.rollNumber << setw(20) << s.name
<< setw(5) << s.age << "\n";
    }
    cout << "----------------------------------------\n";
}

// 3. Requirement: Search for a student by Roll Number.
void searchStudent(const string& filename) {
    string searchRoll;
    cout << "Enter Roll Number to search: ";
    cin >> searchRoll;
    searchRoll = trim(searchRoll);

    vector<Student> students = loadStudentsFromFile(filename);
    bool found = false;

    for (const auto& s : students) {
        if (s.rollNumber == searchRoll) {
            cout << "\nStudent Found:\n";
            cout << "Roll Number: " << s.rollNumber << "\n";
            cout << "Name:        " << s.name << "\n";
            cout << "Age:         " << s.age << "\n";
            found = true;
            break;
        }
    }

    if (!found) {
        cout << "Student not found\n";
    }
}

// 4. Requirement: Update a student's information permanently.
void updateStudent(const string& filename) {
    string targetRoll;
    cout << "Enter Roll Number to update: ";
    cin >> targetRoll;
    targetRoll = trim(targetRoll);
```

```
    vector<Student> students = loadStudentsFromFile(filename);
    bool found = false;

    for (auto& s : students) {
        if (s.rollNumber == targetRoll) {
            found = true;
            cout << "Student found. Enter updated details:\n";
            cout << "Enter New Name: ";
            cin.ignore(); // Clear input buffer
            getline(cin, s.name);
            cout << "Enter New Age: ";
            cin >> s.age;

            saveStudentsToFile(filename, students);
            cout << "Student information updated successfully.\n";
            break;
        }
    }

    if (!found) {
        cout << "Student not found\n";
    }
}

// 5. Requirement: Delete a student's record permanently.
void deleteStudent(const string& filename) {
    string targetRoll;
    cout << "Enter Roll Number to delete: ";
    cin >> targetRoll;
    targetRoll = trim(targetRoll);

    vector<Student> students = loadStudentsFromFile(filename);
    bool found = false;

    for (auto it = students.begin(); it != students.end(); ++it) {
        if (it->rollNumber == targetRoll) {
            students.erase(it);
            found = true;
            saveStudentsToFile(filename, students);
            cout << "Student record deleted successfully.\n";
            break;
        }
    }

    if (!found) {
        cout << "Student not found\n";
    }
```

```
}

// 7. Requirement: Menu-driven system using a loop and switch
statement.
int main() {
    const string filename = "student.txt";
    int choice;

    do {
        cout << "\n===== STUDENT MANAGEMENT SYSTEM =====\n";
        cout << "1. Display All Students\n";
        cout << "2. Search Student\n";
        cout << "3. Update Student\n";
        cout << "4. Delete Student\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";

        // Handle non-integer inputs safely
        if (!(cin >> choice)) {
            cout << "Invalid input. Please enter a number between 1
and 5.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (choice) {
            case 1:
                displayAllStudents(filename);
                break;
            case 2:
                searchStudent(filename);
                break;
            case 3:
                updateStudent(filename);
                break;
            case 4:
                deleteStudent(filename);
                break;
            case 5:
                cout << "Exiting program. Goodbye!\n";
                break;
            default:
                cout << "Invalid choice! Please select an option from
1 to 5.\n";
        }
    } while (choice != 5);
```

```
    return 0;
}
```

How to Seed Test Data

If you need to quickly pre-populate your student.txt file with the assignment's exact sample data, create a text file named student.txt in the exact same directory as your compiled program, and paste the following rows inside it:

```
202312051, lithacynthia, 18
202312052, dessama, 18
202312053, david, 20
```

Key Logic Breakdown

- **Data Persistence:** By loading records into a vector at runtime and overwriting student.txt whenever changes occur, the program ensures your file updates cleanly without data corruption or lingering trailing artifacts.
- **Robust String Trimming:** The custom trim() function strips out excess whitespace surrounding commas when parsing lines out of student.txt. This keeps your text queries clean and makes searching for a roll number bulletproof.